

Keeping context clean by blocking what Claude should not read

There is no .claudeignore

Claude Code has no `.claudeignore` file in the `.gitignore` mold. Blocking file access goes through `permissions.deny` in `.claude/settings.json`, using direct path patterns such as `Read(*.env*)`. The `file_path:` qualifier is invalid for Read, Edit and Write permission rules.

```
{
  "permissions": {
    "deny": [
      "Read(*.env*)",
      "Edit(*.env*)",
      "Write(*.env*)",
      "Read(*.pem)",
      "Read(*.key)",
      "Read(*credentials*)",
      "Bash(cat *.env*)"
    ]
  }
}
```

Why it matters

`deny` has the highest precedence: no `allow` or `ask` rule, at any scope, can override it. The rule blocks `Read`, `Edit`, `Write` calls, and Bash commands that explicitly match a deny pattern. It does not block generic Bash commands without a dedicated deny rule, nor an `ls *.env*` that reveals the file exists without exposing its contents.

What permissions.deny covers

Vector	Blocked
<code>Read()</code>	Yes
<code>Edit()</code> / <code>Write()</code> with a rule	Yes
<code>Bash(cat *.env)</code> with explicit rule	Yes
Generic Bash without a deny rule	No
<code>ls *.env*</code> (filename)	Partial

For defense-in-depth: store secrets outside the project directory, add a PreToolUse hook as a second lock, never commit secrets even "blocked" ones.

.gitignore vs permissions.deny

Mechanism	Role	Scope
<code>.gitignore</code>	Excludes from version control	Git
<code>permissions.deny</code>	Blocks reading/editing/writing via tools	Claude Code
<code>respectGitignore</code> (default: <code>true</code>)	The <code>@</code> picker respects <code>.gitignore</code> patterns	Claude Code

These three mechanisms are independent. A file can be ignored by Git without being blocked for Claude, and the reverse.

Recommended .claude/ folder structure

```
.claude/
├── settings.json           # Team config
├── (committed)
├── settings.local.json    # Local config
├── (gitignore)
├── agents/                # Custom agents
├── skills/                # Skills (includes
│   └── slash commands since CC 2.1.3)
├── hooks/                 # Automation scripts
└── rules/                 # Shared conventions,
    └── optionally path-scoped
```

The `agents/`, `skills/`, `hooks/` and `rules/` folders are committed with the project. `settings.local.json` and the project-root `CLAUDE.local.md` hold local configuration and personal instructions. Auto-memory lives under `~/.claude/projects/<project>/memory/`, separately from this tree.

Clean workspace checklist

- `*.env*`, `*.pem`, `*.key`, `*credentials*` in `permissions.deny`
- Deny rules covering Read, Edit, Write AND the equivalent Bash commands
- Secrets stored outside the project directory where possible
- `settings.local.json` in `.gitignore`
- PreToolUse hook as a second lock on critical files